Lab Assignment 04

BRAC UNIVERSITY

Inspiring Excellence

Course Code:	**CSE111**
Course Title:	**Programming Language II**
Topic:	**Reference Passing, Multiclass Problem & Encapsulation**
Number of Tasks:	**13 (Classwork: 06, Homework: 07)**

[Submit all the Coding Tasks (Homework: Task 1 to 4) in the Google Form shared on buX before the next lab. Submit the Tracing Tasks (Homework: Task 5 to 6) handwritten to your Lab Instructors at the beginning of the lab]

[You are not allowed to change the driver codes of any of the tasks]

CLASSWORK

Task 1

Design the **Shop** class with necessary properties so that the following output is produced.

Driver Code	Output

`public class ShopTester{`
`  public static void main(String[] args){`
`  Shop s1 = new Shop("Agora", 4);`
`  Shop s2 = new Shop(3);`
`  System.out.println("1==========");`
`  s1.info();`
`  System.out.println("2==========");`
`  s2.info();`
`  System.out.println("3==========");`
`  s1.addItem("KitKat", 25);`
`  s1.addItem("Hershey's", 100);`
`  s1.addItem("Dairy Milk", 150);`
`  s1.addItem("Feastables", 500);`
`  s1.addItem("Prime", 1200);`
`  System.out.println("4==========");`
`  s1.info();`
`  System.out.println("5==========");`
`  s1.purchase("KitKat");`
`  s1.purchase("Snickers");`
`  System.out.println("6==========");`
`  s2.info();`
`  System.out.println("7==========");`
`  s2.addItem(s1, "KitKat");`
`  s2.addItem("Toblerone", 80);`
`  s2.addItem(s1, "Mars");`
`  System.out.println("8==========");`
`  s2.info();`
`  }`
`}`
	`Agora shop created!`
	`Mega shop created!`
	`1==========`
	`Shop Name: Agora`
	`Item Details:`
	`No items in shop`
	`2==========`
	`Shop Name: Mega`
	`Item Details:`
	`No items in shop`
	`3==========`
	`Prime, could not be added`
	`4==========`
	`Shop Name: Agora`
	`Item Details:`
	`4 / 4`
	`KitKat - 25 Tk`
	`Hershey's - 100 Tk`
	`Dairy Milk - 150 Tk`
	`Feastables - 500 Tk`
	`5==========`
	`Purchase Complete!`
	`Snickers, is not available in this shop`
	`6==========`
	`Shop Name: Mega`
	`Item Details:`
	`No items in shop`
	`7==========`
	`Mars, not found in Agora!`
	`8==========`
	`Shop Name: Mega`
	`Item Details:`
	`2 / 3`
	`KitKat - 25 Tk`
	`Toblerone - 80 Tk`

Task 2

Write the **Teacher** and **Course** classes so that the TestTeacher class produces the outputs given.
Hint: A teacher can add a maximum of 3 courses.

[The attribute of Course class must be Private]

Driver Code	Output

`public class TestTeacher{`
`  public static void main(String [] args){`
`  Teacher t1 = new Teacher("Matin Saad Abdullah","MSA");`
`  Teacher t2 = new Teacher("Mumit Khan","MMK");`
`  Teacher t3 = new Teacher("Sadia Hamid Kazi","SKZ");`
`  Course c1 = new Course("CSE 110");`
`  Course c2 = new Course("CSE 111");`
`  Course c3 = new Course("CSE 220");`
`  Course c4 = new Course("CSE 221");`
`  Course c5 = new Course("CSE 230");`
`  Course c6 = new Course("CSE 310");`
`  Course c7 = new Course("CSE 320");`
`  Course c8 = new Course("CSE 340");`
`  t1.addCourse(c1);`
`  t1.addCourse(c2);`
`  t2.addCourse(c3);`
`  t2.addCourse(c4);`
`  t2.addCourse(c5);`
`  t3.addCourse(c6);`
`  t3.addCourse(c7);`
`  t3.addCourse(c8);`
`  System.out.println("1=======================");`
`  t1.printDetail();`
`  System.out.println("2=======================");`
`  t2.printDetail();`
`  System.out.println("3=======================");`
`  t3.printDetail();`
`  }`
`}`
	`A new teacher has been created`
	`A new teacher has been created`
	`A new teacher has been created`
	`1=======================`
	`Name: Matin Saad Abdullah`
	`Initial: MSA`
	`List of courses:`
	`CSE 110`
	`CSE 111`
	`2=======================`
	`Name: Mumit Khan`
	`Initial: MMK`
	`List of courses:`
	`CSE 220`
	`CSE 221`
	`CSE 230`
	`3=======================`
	`Name: Sadia Hamid Kazi`
	`Initial: SKZ`
	`List of courses:`
	`CSE 310`
	`CSE 320`
	`CSE 340`

Task 3

Design the **Team** and **Player** classes to get the output as shown.
Hints:

- Create an array in the Team class to store the Player's object
- Use constructor overloading technique for Team class
- A team can add maximum 11 players
- **All attributes of Player class must be kept Private.**

Driver Code	Output

`public class TeamTester {`
`  public static void main(String[] args) {`
`  Team b = new Team();`
`  b.updateName("Bangladesh");`
`  Player mashrafi =  new Player("Mashrafi", 42, 100);`
`  b.addPlayer(mashrafi);`
`  Player tamim =  new Player("Tamim", 35, 70);`
`  b.addPlayer(tamim);`
`  System.out.println("==========");`
`  b.printDetail();`
`  System.out.println("==========");`
`  Team a = new Team("Australia");`
`  Player ponting =  new Player("Ponting", 50, 300);`
`  a.addPlayer(ponting);`
`  Player lee =  new Player("Lee", 49, 200);`
`  a.addPlayer(lee);`
`  a.printDetail();`
`  }`
`}`
	`==========`
	`Team: Bangladesh`
	`List of players:`
	`Name: Mashrafi`
	`Age: 42, Total Matches: 100`
	`Name: Tamim`
	`Age: 35, Total Matches: 70`
	`==========`
	`Team: Australia`
	`List of players:`
	`Name: Ponting`
	`Age: 50, Total Matches: 300`
	`Name: Lee`
	`Age: 49, Total Matches: 200`

Task 4

Design the required class/es so that the following output is generated. Read the following description:

1. You may assume that to board a bus, a student must have the bus pass, and his/her location must match the route of the bus.
2. Additionally, the default maximum capacity of the bus is 2.
3. The location attribute of the Student class will be **private**

Driver Code	Output

public class BracuStudentTester {
public static void main(String[] args) {
BracuStudent st1 = new BracuStudent("Afif", "Mirpur");
System.out.println("1==========");
BracuStudent st2 = new BracuStudent("Shanto",
"Motijheel");
BracuStudent st3 = new BracuStudent("Taskin", "Mirpur");
st1.showDetails();
st2.showDetails();
System.out.println("2==========");
st3.showDetails();
System.out.println("3==========");
BracuBus bus1 = new BracuBus("Mirpur");
BracuBus bus2 = new BracuBus("Azimpur", 5);
bus1.showDetails();
bus2.showDetails();
System.out.println("4==========");
st2.collectPass();
st3.collectPass();
System.out.println("5==========");
st2.showDetails();
st3.showDetails();
System.out.println("6==========");
bus1.board();
System.out.println("7==========");
bus1.board(st1, st2);
System.out.println("8==========");
st1.collectPass();
st2.setLocation("Mirpur");
st1.showDetails();
st2.showDetails();
System.out.println("9==========");
bus1.board(st1);
bus1.board(st2, st3);
System.out.println("10==========");
bus1.showDetails();
}
}
	1==========
	Student Name: Afif
	Lives in Mirpur
	Have Bus Pass? false
	Student Name: Shanto
	Lives in Motijheel
	Have Bus Pass? false
	2==========
	Student Name: Taskin
	Lives in Mirpur
	Have Bus Pass? false
	3==========
	Bus Route: Mirpur
	Passenger Count: 0 (Max: 2)
	Passengers on Board:
	Bus Route: Azimpur
	Passenger Count: 0 (Max: 5)
	Passengers on Board:
	4==========
	5==========
	Student Name: Shanto
	Lives in Motijheel
	Have Bus Pass? true
	Student Name: Taskin
	Lives in Mirpur
	Have Bus Pass? true
	6==========
	No passengers
	7==========
	You don't have a bus pass!
	You got on the wrong bus!
	8==========
	Student Name: Afif
	Lives in Mirpur
	Have Bus Pass? true
	Student Name: Shanto
	Lives in Mirpur
	Have Bus Pass? true
	9==========
	Afif boarded the bus.
	Shanto boarded the bus.
	Bus is full!
	10==========
	Bus Route: Mirpur
	Passenger Count: 2 (Max: 2)
	Passengers on Board:
	Afif Shanto

Task 5

```
public class Trace {
 public int sum, temp;
 public Trace(int sum, int temp){
   this.sum = sum;
   this.temp = temp;
 }
}
class Quiz5{
 public int sum = 11, x = -2, y = 16;
 public Trace trace = null;
 public Quiz5(Trace t){
   trace = t;
   int x = trace.temp + y;
   sum = sum + (t.sum++) + y;
   System.out.println(trace.sum + " " + sum + " " + x);
   sum -= 10;
 }
 public void methodA(int sum, int temp){
   sum = -3 + this.sum - trace.sum;
   x = sum + 13 + y;
   y = trace.temp + temp + sum;
   this.sum = y + methodB(trace.temp, trace) + trace.temp;
   System.out.println(sum + " " + y + " " + this.sum);
 }
 public int methodB(int x, Trace temp){
   int sum = x + temp.sum + this.x;
   temp.temp = sum + this.sum;
   System.out.println(x + " " + temp.temp + " " + sum);
   return sum;
 }
}
```

```
Trace p = new Trace(3, 4);
Quiz5 q = new Quiz5(p);
q.methodA(4, 8);
q.methodA(5, 10);
```

Output		

Task 6

```
public class TracingX {
  public int x, y = 1;
  public int metA(int y){
    y += x + 3;
    int temp = y + this.y;
    if (temp % 2 == 0){
      return temp;
    }
    TracingX t = new TracingX();
    t.y = this.x - (++x) + t.x;
    this.y = y + t.metA(t.x);
    System.out.println(x +" "+ y +" "+temp);
    return temp+this.y;
  }
}
```

Driver code:

```
public class TesterX {
 public static void main(String[] args) {
   TracingX t1 = new TracingX();
   t1.y = t1.x = 5;
   TracingX t2 = new TracingX();
   t2.x = t1.metA(2);
   t2.y = t2.metA(4);
   System.out.println(t1.y +t1.x +" "+t2.x +" "+t2.y);
 }
}
```

Output:

HOMEWORK

Task 1

Design the **Triangle** Class that will produce the following output. We will consider both triangles to have the same sides if all sides are equal in the same orientation/sequence only.

Types of Triangle:

- Equilateral: When all sides in the same orientation are equal.
- Isosceles: When any two sides of a triangle in the same orientation are equal.
- Scalene: When all sides are of different lengths.

Driver Code

```
public class TriangleTester{
 public static void main(String args[]){
   Triangle t1 =  new Triangle(4, 4, 4);
   Triangle t2 =  new Triangle(4, 5, 6);
   Triangle t3 =  new Triangle(4, 5, 6);
   Triangle t4 =  new Triangle(5, 4, 6);

   t1.triangleDetails();
   System.out.println("--------1--------");
   System.out.println(t1.printTriangleType());
   System.out.println("--------2--------");
   t3.triangleDetails();
   System.out.println(t3.printTriangleType());
   System.out.println("--------3--------");
   t4.triangleDetails();
   System.out.println(t4.printTriangleType());
   System.out.println("--------4--------");
   t2.compareTriangles(t3);
   System.out.println("--------5--------");
   t1.compareTriangles(t2);
   System.out.println("--------6--------");
   t1 = t2;
   t1.compareTriangles(t2);
   System.out.println("--------7--------");
   t3.compareTriangles(t4);
 }
}
```

Output

```
Three sides of the triangle are: 4, 4, 4
Perimeter: 12
--------1--------
This is an Equilateral Triangle.
--------2--------
Three sides of the triangle are: 4, 5, 6
Perimeter: 15
This is a Scalene Triangle.
--------3--------
Three sides of the triangle are: 5, 4, 6
Perimeter: 15
This is a Scalene Triangle.
--------4--------
Addresses are different but the sides of the triangles are equal.
--------5--------
Addresses, length of the sides and perimeter all are different.
--------6--------
These two triangle objects have the same address.
--------7--------
Only the perimeter of both triangles is equal.
```

Task 2

Design the **Player** class to obtain the following output from the driver code. You can assume, each player can defeat at most 5 villains.

Driver Code

```
public class PlayerTester {
  public static void main(String[] args) {
    Player ben = new Player("Ben", 10);
    System.out.println("======1======");
    ben.viewInfo();
    System.out.println("======2======");
    ben.defeatVillain("Vilgax", 100);
    System.out.println("======3======");
    ben.defeatVillain("Yamcha", 10);
    System.out.println("======4======");
    ben.viewInfo();
    System.out.println("======5======");
    ben.defeatVillain('8', "Vilgax", 100);
    System.out.println("======6======");
    ben.viewInfo();
    System.out.println("======7======");
    Player kevin = new Player("Kevin");
    System.out.println("======8======");
    kevin.viewInfo();
    System.out.println("======9======");
    ben.defeatVillain(kevin);
    System.out.println("======10======");
    ben.viewInfo();
    System.out.println("======11======");
    Player goku = new Player("Goku", 9000);
    System.out.println("======12======");
    ben.defeatVillain(goku);
  }
}
```

Expected Output

```
Ben joined the game
HP: 10
======1======
Player Name: Ben
Current HP: 10
======2======
failed to defeat Vilgax
======3======
defeated Yamcha
======4======
Player Name: Ben
Current HP: 20
Defeated:
Yamcha,
======5======
HP with 8x boost: 160
defeated Vilgax
======6======
Player Name: Ben
Current HP: 260
Defeated:
Yamcha, Vilgax,
======7======
Kevin joined the game
HP: 100
======8======
Player Name: Kevin
Current HP: 100
======9======
defeated Kevin
======10======
Player Name: Ben
Current HP: 360
Defeated:
Yamcha, Vilgax, Kevin,
======11======
Goku joined the game
HP: 9000
======12======
failed to defeat Goku
```

Task 3

Design the **Student** and the **Connect** class so that the following output is produced.
Note:

- A student's email, password, and login status are null by default while creating an object of the Student class.
- The password and login status attributes of the Student class will be **private**
- Your code should satisfy the conditions mentioned in the output only.
- Connect class will have two instance variables: totalAdvisee and an array of Student type to store the student object. The array will be updated inside the advising() method only when the advising is successful.
- Connect can take at most 5 advisees.

Driver Code

```
public class ConnectTester {
  public static void main(String[] args) {
    Student rakib = new Student("Rakib", 12301455,
"CSE");
    Student roy = new Student("Roy", 12501345, "CS");
    System.out.println("1*************");
    Connect connectObj = new Connect();
    System.out.println("2*************");
    connectObj.login(rakib);
    System.out.println("3*************");
    connectObj.advising(rakib);
    System.out.println("4*************");
    rakib.email = "rakib@hotmail.com";
    rakib.setPassword("1234");
    System.out.println("5*************");
    connectObj.login(rakib);
    System.out.println("6*************");
    connectObj.advising(rakib);
    System.out.println("7*************");
    connectObj.advising(rakib, "CSE110", "PHY111",
"MAT110", "CSE260");
    System.out.println("8*************");
    connectObj.advising(rakib, "CSE110",
"PHY111","MAT110");
    System.out.println("9*************");
    connectObj.allAdviseeInfo();
    System.out.println("10*************");
    roy.email = "roy@hotmail.com";
    roy.setPassword("abcd");
    connectObj.login(roy);
    System.out.println("11*************");
    connectObj.advising(roy, "CSE110", "ENG101",
"PHY112");
    System.out.println("12*************");
    connectObj.allAdviseeInfo();
  }
}
```

Expected Output

```
Student object is created
Student object is created
1*************
Connect is ready to use!
2*************
Email and password need to be set.
3*************
Please login to advise courses!
4*************
5*************
Login successful
6*************
You haven't selected any courses.
7*************
You need special approval to take
more than 3 courses.
8*************
Advising successful!
9*************
Total Advisee: 1
Name: Rakib ID: 12301455
Department: CSE
Advised Courses:
CSE110 PHY111 MAT110
==============
10*************
Login successful
11*************
Advising successful!
12*************
Total Advisee: 2
Name: Rakib ID: 12301455
Department: CSE
Advised Courses:
CSE110 PHY111 MAT110
==============
Name: Roy ID: 12501345
Department: CS
Advised Courses:
CSE110 ENG101 PHY112
==============
```

Task 4

Spaceship: This class represents a spaceship. Each spaceship has a **name** and a **capacity** (the maximum weight it can carry).
Cargo: This class represents a piece of cargo. Each cargo item has a **name** and a **weight**. Both attributes should be **private** which means they cannot be accessed directly from outside of the class.
A Spaceship contains Cargo. That means each spaceship can carry multiple cargo items, but the total weight of the cargo cannot exceed the spaceship's capacity. Also, the maximum number of cargo items is 100. Your task is to design the **Spaceship** and **Cargo** class with necessary properties so that the given output is produced for the provided driver code.

Driver Code

```
public class ShipTester {
 public static void main(String[] args) {
  Spaceship falcon = new Spaceship("Falcon", 50000);
  Spaceship apollo = new Spaceship("Apollo", 100000);
  Spaceship enterprise = new
Spaceship("Enterprise",220000);
  System.out.println("1.===========");
  Cargo gold = new Cargo("Gold", 20000);
  Cargo platinum = new Cargo("Platinum", 25000);
  Cargo dilithium = new Cargo("Dilithium", 50000);
  Cargo trilithium = new Cargo("Trilithium", 70000);
  Cargo neutronium = new Cargo("Neutronium", 80000);
  System.out.println("2.===========");
  falcon.loadCargo(gold);
  falcon.loadCargo(platinum);
  falcon.displayDetails();
  System.out.println("3.===========");
  apollo.loadCargo(gold);
  apollo.displayDetails();
  System.out.println("4.===========");
  falcon.loadCargo(neutronium);
  System.out.println("5.===========");
  enterprise.loadCargo(dilithium);
  enterprise.loadCargo(trilithium);
  enterprise.loadCargo(neutronium);
  enterprise.displayDetails();
 }
}
```

Output

```
1.===========
2.===========
Spaceship Name: Falcon
Capacity: 50000
Current Cargo Weight: 45000
Cargo:Gold Platinum
3.===========
Spaceship Name: Apollo
Capacity: 100000
Current Cargo Weight: 20000
Cargo:Gold
4.===========
Warning: Unable to load Neutronium
inside Falcon. Exceeds capacity by
75000.
5.===========
Spaceship Name: Enterprise
Capacity: 220000
Current Cargo Weight: 200000
Cargo:Dilithium Trilithium
Neutronium
```

Task 5

```
public class TestX{
 public int result = 17, i, n = 7, m = 5;
 public int [] num = {6,8,11};
 public void compute(int [] nums){
   n = nums[0] + this.m;
   this.result += nums[i];
   TestX temp = new TestX();
   boolean check = !(temp.modify(temp, num));
   if (check) {
     m = nums[1] + temp.result;
   } else {
     m = result - nums[3];
   }
   System.out.println(m + " " + n + " " + result);
 }
 public boolean modify(TestX obj, int[] arr){
   arr[1]++;
   i=i+1;
   obj.m = this.result + arr[i];
   obj.result = obj.m - obj.n + n;
   System.out.println(obj.m + " " + obj.n + " " + result);
   return arr[1] % 2 == 0;
 }
 public boolean modify(TestX obj){
   num[i]++;
   i=i+1;
   obj.m = this.result + num[i];
   obj.result = num[2] - obj.n + n;
   System.out.println(obj.m + " " + obj.n + " " + result);
   return num[1] % 2 != 0;
 }
}
```

```
public class tester12{
  public static void main(String[] args){
    TestX tx = new TestX();
    int[] data = {4, 6, 2, 3, 5};
    tx.compute(data);
    tx.modify(tx);
  }
}
```

Outputs		

Task 6

```
1   public class msgClass{
2     public int content;
3   }
4   class FinalT5A{
5     public int sum = 2, y = 1, x = 1;
6     public void methodA(){
7       int x = 6, y = 0;
8       msgClass myMsg = new msgClass();
9       myMsg.content = this.x;
10      x = x + myMsg.content;
11      this.y = this.y + methodB(myMsg, myMsg.content);
12      System.out.println(x + " " + this.y+ " " + sum);
13      y =  this.y/2 + this.x;
14      x = y + sum/2;
15      sum = x + y + myMsg.content;
16      System.out.println(x + " " + y+ " " + sum);
17    }
18    public int methodB(msgClass mg2, int mg1){
19      int x = 0;
20      y = y + mg2.content;
21      mg2.content = y + mg1;
22      x = this.x + 3 + mg1;
23      sum = sum + x + y;
24      System.out.println(this.x + " " + this.y+ " " + sum);
25      mg2.content = sum - mg1 ;
26      return sum;
27    }
28  }
```

DRIVER CODE	OUTPUTS		

`public class Tester5{`
`  public static void main(String args []){`
`  FinalT5A fT5A = new FinalT5A();`
`  fT5A.methodA();`
`  }`
`}`			

Ungraded Tasks (Optional)

(You don't have to submit the ungraded tasks)

Task 1

Design the Company and Employee classes so that the Tester1 class produces the given outputs.

Restriction: Company class can't have more than 1 array and all attributes of Employee class must be Private.

Driver Code

```
public class Tester1{
  public static void main(String args[]){
    Employee e1 = new Employee();
    Employee e2 = new Employee("Alif", 34, "Fulltime");
    System.out.println("1-------------");
    Company c1 = new Company();
    c1.details();
    System.out.println("2-------------");
    Employee e3 = new Employee("Akter", 36,"Part-time");
    Employee e4 = new Employee("Ria", 38, "Fulltime");
    System.out.println("3-------------");
    c1.addEmployee(e2);
    c1.addEmployee(e3);
    System.out.println("4-------------");
    c1.details();
    System.out.println("5-------------");
    c1.addEmployee(e4);
    c1.addEmployee(e1);
    System.out.println("6-------------");
    c1.details();
    System.out.println("7-------------");
    c1.removeEmployee(e4);
    System.out.println("6-------------");
    c1.details();
  }
}
```

Output

```
A default employee has been
created
1-------------
Company Name: ABC Company
Total Employee: 0
Fulltime Employees:
Part-Time Employees:
2-------------
3-------------
Alif has joined the company
Akter has joined the company
4-------------
Company Name: ABC Company
Total Employee: 2
Fulltime Employees:
Name: Alif, ID: 34
Part-Time Employees:
Name: Akter, ID: 36
5-------------
Ria has joined the company
No more vacancy
6-------------
Company Name: ABC Company
Total Employee: 3
Fulltime Employees:
Name: Alif, ID: 34
Name: Ria, ID: 38
Part-Time Employees:
Name: Akter, ID: 36
7-------------
Ria has left the company
6-------------
Company Name: ABC Company
Total Employee: 2
Fulltime Employees:
Name: Alif, ID: 34
Part-Time Employees:
Name: Akter, ID: 36
```

Task 2

Write "**Student**" class to show the following expected outputs

Note:

- A student can't take any course until the CGPA is set.
- A student cannot take more than 4 courses.
- A student with CGPA below 3 cannot take more than 3 courses.

Driver Code	Expected Output

`public class StudentDriver {`
`  public static void main(String[] args){`
`  Student student1 = new Student(12345678);`
`  System.out.println("1----------");`
`  student1.addCourse("CSE110");`
`  System.out.println("2----------");`
`  student1.storeCG(2.5);`
`  student1.addCourse("CSE110");`
`  student1.addCourse("ENG101");`
`  student1.showAdvisee();`
`  System.out.println("3----------");`
`  student1.removeAllCourse();`
`  student1.showAdvisee();`
`  System.out.println("4----------");`
`  student1.storeID(54652365);`
`  String[] courses = {"SOC101","CSE111","ENG102"};`
`  student1.addCourse(courses);`
`  student1.showAdvisee();`
`  System.out.println("5----------");`
`  student1.addCourse("CSE230");`
`  student1.showAdvisee();`
`  System.out.println("6----------");`
`  Student student2 = new Student(975738383,3.7);`
`  System.out.println("7----------");`
`  String[] courses2 =`
`{"CSE220","PHY112","MAT120","BUS101","CHN101"};`
`  student2.addCourse(courses2);`
`  student2.showAdvisee();`
`  }`
`}`
	`A student with ID 12345678 has`
	`been created.`
	`1----------`
	`Failed to add CSE110`
	`Set CG first`
	`2----------`
	`Student ID: 12345678, CGPA: 2.5`
	`Added courses are:`
	`CSE110 ENG101`
	`3----------`
	`Student ID: 12345678, CGPA: 2.5`
	`No courses added.`
	`4----------`
	`Student ID: 54652365, CGPA: 2.5`
	`Added courses are:`
	`SOC101 CSE111 ENG102`
	`5----------`
	`Failed to add CSE230`
	`CG is low. Can't add more than 3`
	`courses.`
	`Student ID: 54652365, CGPA: 2.5`
	`Added courses are:`
	`SOC101 CSE111 ENG102`
	`6----------`
	`A student with ID 975738383 and`
	`cgpa 3.7 has been created.`
	`7----------`
	`Failed to add CHN101`
	`Maximum 4 courses allowed.`
	`Student ID: 975738383, CGPA: 3.7`
	`Added courses are:`
	`CSE220 PHY112 MAT120 BUS101`

Task 3

Design the **necessary** classes for the NetFlix system with the necessary properties to produce the given output for the provided driver code.

Driver code

```
public class NetflixAccount{
  public static void main(String[] args){
    Movie m1 = new Movie("Avengers", "Sci-fi", "1:59:08");
    Movie m2 = new Movie("Lord of the Rings", "Fantasy", "2:58:05");
    Movie m3 = new Movie("Final Destination","Horror");
    Movie m4 = new Movie("Fast and Furious","Action", "2:05:04");
    System.out.println("==========================");
    NetflixUser user1 = new NetflixUser("Martin Freeman", "Free");
    user1.addToFavourites(m1);
    user1.showFavourites();
    user1.upgradePlan();
    user1.addToFavourites(m1);
    user1.showFavourites();
    System.out.println("==========================");
    NetflixUser user2 = new NetflixUser("Vin Diesel", "Premium");
    user2.addToFavourites(m1);
    user2.addToFavourites(m2);
    user2.addToFavourites(m3);
    user2.addToFavourites(m4);
    user2.showFavourites();
    System.out.println("==========================");
    NetflixUser user3 = new NetflixUser("Chris Evans");
    user3.addToFavourites(m1);
    user3.upgradePlan();
    user3.addToFavourites(m2);
    user3.addToFavourites(m3);
    user1.upgradePlan();
  }
}
```

Output

```
==========================
New user account created!
Cannot add movies to favourites.
Please upgrade to Netflix Premium.
No available favourites. Please
upgrade to Netflix Premium.
Welcome to Netflix Premium!
Movie added to favourites.
Showing Martin Freeman's Favourites:
Avengers, Sci-fi, Duration: 1:59:08
==========================
New user account created!
Movie added to favourites.
Movie added to favourites.
Movie added to favourites.
Cannot add new movies. Favourites
list is full.
Showing Vin Diesel's Favourites:
Avengers, Sci-fi, Duration: 1:59:08
Lord of the Rings, Fantasy, Duration:
2:58:05
Final Destination, Horror, Duration:
Unknown
==========================
New user account created!
Cannot add movies to favourites.
Please upgrade to Netflix Premium.
Welcome to Netflix Premium!
Movie added to favourites.
Movie added to favourites.
You already have Netflix Premium!
```

Task 4

Please write the **Student** and **Department** class with the necessary properties so that the provided driver code generates the output given below. The id of the Student class will be **private**. For simplicity, assume that a department can add a maximum of 5 students.

Driver Code

```
public class Tester1 {
 public static void main(String[] args) {
  Student s1 = new Student("Akib", 10, 3.29);
  Student s2 = new Student("Reza", 15, 3.45);
  Student s3 = new Student("Kabir", 20,4.0);
  System.out.println("1==========");
  Department cse = new Department("CSE");
  cse.findStudent(-100);
  System.out.println("2==========");
  cse.addStudent(s1, s2, s3);
  System.out.println("3==========");
  cse.details();
  System.out.println("4==========");
  cse.findStudent(15);
  System.out.println("5==========");
  Student s4 = new Student("Nakib", 15,3.22);
  cse.addStudent(s4);
  System.out.println("6==========");
  s4.setId(25);
  cse.addStudent(s4);
  System.out.println("7==========");
  cse.details();
  System.out.println("8==========");
  Student s5 = new Student("Sakib", 30,2.29);
  cse.addStudent(s5);
  System.out.println("9==========");
  cse.details();
 }
}
```

Output

```
1==========
Student with this ID doesn't exist, Please
give a valid ID
2==========
Welcome to CSE department, Akib
Welcome to CSE department, Reza
Welcome to CSE department, Kabir
3==========
Department Name: CSE
Number of student:3
Details of the students:
Student name: Akib, ID: 10, cgpa: 3.29
Student name: Reza, ID: 15, cgpa: 3.45
Student name: Kabir, ID: 20, cgpa: 4.0
4==========
Student info:
Student Name: Reza
ID: 15
CGPA:  3.45
5==========
Student with the same ID already exists,
Please try with another ID
6==========
Welcome to CSE department, Nakib
7==========
Department Name: CSE
Number of student:4
Details of the students:
Student name: Akib, ID: 10, cgpa: 3.29
Student name: Reza, ID: 15, cgpa: 3.45
Student name: Kabir, ID: 20, cgpa: 4.0
Student name: Nakib, ID: 25, cgpa: 3.22
8==========
Welcome to CSE department, Sakib
9==========
Department Name: CSE
Number of student:5
Details of the students:
Student name: Akib, ID: 10, cgpa: 3.29
Student name: Reza, ID: 15, cgpa: 3.45
Student name: Kabir, ID: 20, cgpa: 4.0
Student name: Nakib, ID: 25, cgpa: 3.22
Student name: Sakib, ID: 30, cgpa: 2.29
```

Task 5

```
pubic class QuizA {
 public int x, y;
 public int sum = 1;
 public QuizA(int x, int y) {
   this.x = y;
   this.y = x;
 }
 public void methodA() {
   int x = 3;
   y = this.y + x;
   QuizA exam = new QuizA(5, 11);
   exam.sum = x;
   exam.y = this.y;
   x = this.x + x + exam.sum;
   this.y = this.sum + methodB(exam.sum, exam);
   System.out.println(exam.x + " " + this.y + " " + sum);
   sum = x % 2 + this.x;
   y = x + y + exam.sum;
   System.out.println(x + " " + y + " " + sum);
 }
 public int methodB(int x1, QuizA x2) {
   int y = 0;
   y = this.y + x2.sum;
   x2.sum = x1 + x2.x;
   sum = sum + x + y;
   System.out.println(this.x + " " + this.y + " " + sum);
   return x2.sum;
 }
}
```

Driver Code	Output		

`public class QuizTesterA{`
`  public static void main(String []args){`
`  QuizA q1 = new QuizA(3,4);`
`  q1.methodA();`
`  }`
`}`			

Task 6

```
public class Trace{
  public int x, y = 5, temp = -4, sum = 3;
  public Trace(){
    y = temp + 3 ;
    sum = 3 + temp + 2;
    temp -= 2;
  }
  public Trace(Trace t){
    sum = t.sum++;
    x = t.x + 2;
    t.methodB(2,3);
  }
  public void methodA(int m, int n){
    int x = 2;
    y = y + m + (temp++);
    x = x + 5 +  n;
    sum = sum + x + y;
    System.out.println(x + " " + y+ " " + sum);
  }
  public void methodB(int m, int n){
    int  y = 0;
    y = y + this.y;
    x = this.y + 2 + temp;
    methodA(x, y);
    sum = x + y + sum;
    System.out.println(x + " " + y+ " " + sum);
  }
}
```

```
public class TraceTester {
   public static void main(String args []){
    Trace t1 = new Trace();
    Trace t2 = new Trace(t1);
    t1.methodA(1, 2);
    t2.methodB(3, 2);
   }
}
```

Outputs		

Task 7

```
public class A {
 public int x, y = 2, sum = 5;
 public A(){
   x = sum - y;
   this.methodB(1);
 }
 public A(int x){
   this.x = x;
 }
 public void methodB(int y) {
   int temp = (y++) + (++this.y);
   sum = y + temp - ++this.x;
   System.out.println(y + " " + temp + " " + (sum++));
   this.y = (++temp) + methodC(temp, y) + x;
 }
 public void methodA(int temp, int x) {
   A a2 = new A(6);
   a2.sum = this.methodC(a2);
   System.out.println(this.x + " " + a2.sum + " " + y);
   this.sum = a2.methodC(this);
   System.out.println(a2.x + " " + this.sum + " " + a2.y);
 }
 public int methodC(A a) {
   return this.x + a.y + a.sum;
 }
 public int methodC(int sum, int y) {
   y = (this.x++) + sum + 3;
   return y;
 }
}
```

```
public class TesterA{
  public static void main(String [] args) {
    A a1 = new A();
    a1.methodA(4, 3);
  }
}
```

Outputs		